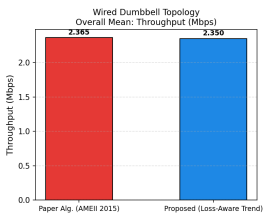
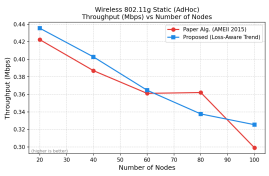
Loss-Aware, Trend-Based Adaptive RTO

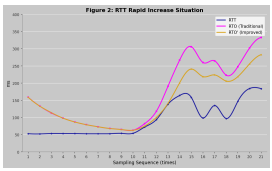
for TCP Retransmission Timeout

NS-3 Simulation Project Report

Student Name	Dhruba Kishore Nag
Student ID	2105062
Department	Computer Science and Engineering
Institution	Bangladesh University of Engineering and Technology (BUET)
Supervisor	Ishrat Jahan Maam
Supervisor Affiliation	Department of CSE, BUET
Date	April 2026







Simulator: ns-3.45 Language: C++17 / Python 3

Contents

1 Project Proposal: Brief Introduction **4**

1.1 Motivation 4

1.2 Problem Statement 4

2 Background: The Paper Algorithm (AMEII 2015) **5**

2.1 Algorithm Description 5

2.2 Online Reproduction Results 5

3 Proposed Solution and its Logic **7**

3.1 Limitations of the Paper Algorithm 7

3.2 Proposed Modifications 7

3.2.1 Windowed Trend k_{trend} 7

3.2.2 Loss-Aware RTO Scaling 8

3.2.3 Logical Rationale 8

3.3 Proposed Algorithm: Comparison Graphs 8

4 Classes Implemented in the Simulator **11**

4.1 `ns3::RttImprovedEstimator` (`contrib/improved-rto2/model/`) 11

4.2 `ns3::RttProposed` (`src/improved-rto/model/`) 12

4.3 Simulation Scripts 13

4.3.1 `scratch/online/paper-rto-dual.cc` 13

4.3.2 `scratch/proposed/paper-rto-proposed.cc` 13

4.3.3 `scratch/submission_checklist/wired/wired-sim.cc` 13

4.3.4 `scratch/submission_checklist/wireless/wireless-sim.cc` . . 13

5 Network Topologies Under Simulation **14**

5.1 Wired Dumbbell Topology 14

5.2 Wireless 802.11g Ad-Hoc Static Topology 15

6 Parameters Under Variation **16**

7 Modifications Made in the Simulator **17**

7.1 New RTO Estimator Module: `src/improved-rto/` 17

7.2 New Contrib Module: `contrib/improved-rto2/` 17

7.3 Algorithm Selection in Simulation Scripts . . . 17
7.4 Wireless: UDP instead of TCP . . . 18
7.5 CSV Append Mode . . . 18

8 Results with Graphs **19**
8.1 Wired Topology: Effect of Number of Nodes . . . 19
8.2 Wired Topology: Effect of Number of Flows . . . 20
8.3 Wired Topology: Effect of Packets per Second . . . 21
8.4 Wired Topology: Overall Summary . . . 22
8.5 Wireless Topology: Effect of Number of Nodes . . . 23
8.6 Wireless Topology: Effect of Number of Flows . . . 24
8.7 Wireless Topology: Effect of Packets per Second . . . 25
8.8 Wireless Topology: Effect of Coverage Area . . . 26
8.9 Wireless Topology: Overall Summary . . . 27
8.10 Post-Evaluation Changes and Observations . . . 28

9 Summary of Findings **29**
9.1 Wired Topology Findings . . . 29
9.2 Wireless Topology Findings . . . 30

10 Discussion **31**
10.1 Validity of the Online Reproduction . . . 31
10.2 Effect of Windowed Trend (W) . . . 31
10.3 Loss-Aware Scaling (γ) . . . 31
10.4 Wired vs. Wireless Discrepancy . . . 32
10.5 Limitations and Future Work . . . 32
10.6 Conclusion . . . 33

A Raw Data Tables **34**
A.1 Wired Simulation Results (Selected Rows) . . . 34
A.2 Wireless Simulation Results (Selected Rows) . . . 35

List of Figures

2.1 Online reproduction of AMEII 2015 paper figures: traditional (Jacobson) vs. improved algorithm. 6
2.2 Mean absolute error from paper Table 1 (online reproduction). 6

3.1 Proposed algorithm (W=1) vs. paper algorithm. 8
3.2 Effect of window size W on RTO during the increasing-RTT phase. 9
3.3 Effect of window size W on RTO during the decreasing-RTT phase. 9
3.4 Proposed algorithm summary across window sizes $W \in \{1, 2, 3, 4\}$. 10

5.1 Wired dumbbell topology. $N/2$ senders on the left, $N/2$ receivers on the right; R_1–R_2 is the shared bottleneck link. 14

8.1 Wired topology: varying number of nodes (flows=30, pps=300). 19
8.2 Wired topology: varying number of flows (nodes=60, pps=300). 20
8.3 Wired topology: varying packets per second (nodes=60, flows=30). 21
8.4 Wired topology: overall mean metric comparison (Paper vs. Proposed). . . 22
8.5 Wired summary table: Paper vs. Proposed mean metrics. 22
8.6 Wireless topology: varying number of nodes (flows=30, pps=300, coverage=750 m). 23
8.7 Wireless topology: varying number of flows (nodes=60, pps=300, coverage=750 m). 24
8.8 Wireless topology: varying pps (nodes=60, flows=30, coverage=750 m). . . 25
8.9 Wireless topology: varying coverage area (nodes=60, flows=30, pps=300). 26
8.10 Wireless topology: overall mean metric comparison (Paper vs. Proposed). . 27
8.11 Wireless summary table: Paper vs. Proposed mean metrics. 27

Chapter 1

Project Proposal: Brief Introduction

1.1 Motivation

TCP's Retransmission Timeout (RTO) is a fundamental congestion control mechanism. The classic Jacobson/Karels algorithm (RFC 6298) computes:

$$\mathrm{SRTT} = (1-\alpha)\,\mathrm{SRTT} + \alpha\,\mathrm{RTT} \tag{1.1}$$

$$\mathrm{RTTVAR} = (1-\beta)\,\mathrm{RTTVAR} + \beta\,|\mathrm{SRTT} - \mathrm{RTT}| \tag{1.2}$$

$$\mathrm{RTO} = \mathrm{SRTT} + 4 \cdot \mathrm{RTTVAR} \tag{1.3}$$

with fixed $\alpha = \frac{1}{8}$ and $\beta = \frac{1}{4}$. Fixed smoothing factors are unable to react well to rapidly changing network conditions. When RTT is increasing (network becoming congested), a sluggish α causes SRTT to lag the true RTT, resulting in an under-estimated RTO that fires too early and wastes bandwidth with spurious retransmissions. Conversely when RTT is falling, a large RTTVAR inflates the RTO unnecessarily, stalling recovery.

1.2 Problem Statement

This project investigates whether RTO accuracy can be improved by:

1. Adapting α and β dynamically to the *rate of RTT change* (as proposed by Xiao and Zhang, AMEII 2015), and
2. Further extending the adaptation with a *windowed trend* over the last W RTT samples and a *loss-aware* RTO scaling factor γ.

The experiments are conducted in the ns-3.45 discrete-event network simulator over both wired (dumbbell P2P) and wireless (802.11g Ad-Hoc) topologies.

Chapter 2

Background: The Paper Algorithm (AMEII 2015)

2.1 Algorithm Description

Xiao Jianliang and Zhang Kun (AMEII 2015) proposed replacing the fixed α, β in Jacobson's algorithm with values that adapt to the instantaneous rate of RTT change k:

$$k = \frac{|\mathrm{RTT}_{n+1} - \mathrm{RTT}_n|}{\mathrm{RTT}_n}, \quad k \in [0, 1] \tag{2.1}$$

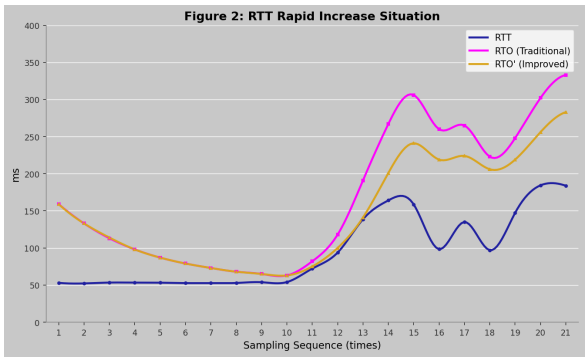
$$\alpha = \alpha_0 \cdot (1 + k) \tag{2.2}$$

$$\beta = \beta_0 \cdot (1 - k) \tag{2.3}$$

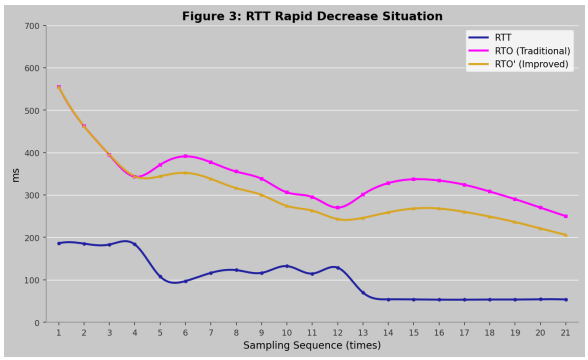
where $\alpha_0 = \frac{1}{8}$ and $\beta_0 = \frac{1}{4}$. When the RTT is changing rapidly ($k \to 1$), α increases so SRTT tracks faster, while β decreases to stabilise RTTVAR. SRTT and RTTVAR are then updated exactly as in Jacobson (Equations 1.1–1.2), and RTO is computed by Equation 1.3.

2.2 Online Reproduction Results

To validate the implementation, an *online* (in-simulation) reproduction of the paper's figures was performed. A single dumbbell topology with $N_{\text{flows}} = 9$ was used. Flows were added one-by-one every 10 s (increasing RTT phase, 0–90 s) and then removed one-by-one (decreasing RTT phase, 90–180 s). At 21 evenly spaced checkpoints per phase the averaged raw RTT was fed into both a standard Jacobson estimator and the improved estimator; RTO was read after each update.



(a) RTO vs. RTT: increasing-load phase



(b) RTO vs. RTT: decreasing-load phase

Figure 2.1: Online reproduction of AMEII 2015 paper figures: traditional (Jacobson) vs. improved algorithm.

Table 1: The Percentage Gamma (γ) of RTT Variety

Seq	RTT Rapid Increase Traditional (γ)	RTT Rapid Increase Improved (γ')	RTT Rapid Decrease Traditional (γ)	RTT Rapid Decrease Improved (γ')
1	200.00%	200.00%	198.56%	198.56%
2	154.75%	154.75%	149.50%	149.50%
3	111.76%	113.64%	115.22%	115.76%
4	83.83%	83.83%	86.95%	87.49%
5	63.53%	63.53%	244.68%	219.60%
6	50.07%	50.07%	303.39%	263.15%
7	38.73%	38.73%	224.45%	190.89%
8	28.76%	28.76%	189.05%	157.30%
9	20.76%	20.76%	190.82%	158.12%
10	16.63%	16.63%	131.43%	107.23%
11	13.57%	3.88%	157.50%	129.57%
12	25.88%	6.68%	109.78%	88.80%
13	37.71%	1.66%	331.65%	252.78%
14	62.72%	22.50%	508.55%	380.53%
15	92.91%	51.93%	528.12%	399.52%
16	164.10%	122.45%	528.80%	404.55%
17	96.40%	66.01%	511.19%	390.46%
18	129.72%	112.20%	476.31%	365.91%
19	68.37%	48.68%	443.36%	342.18%
20	63.82%	38.86%	398.99%	308.43%
21	81.17%	53.97%	367.17%	284.94%
Avg	**76.44%**	**61.88%**	**295.02%**	**237.87%**

Figure 2.2: Mean absolute error from paper Table 1 (online reproduction).

Chapter 3

Proposed Solution and its Logic

3.1 Limitations of the Paper Algorithm

While the AMEII 2015 algorithm improves upon Jacobson, it has two weaknesses:

1. **Single-sample trend:** k is computed from only two consecutive RTT samples, making it sensitive to transient noise.
2. **No loss awareness:** Packet loss (a strong indicator of congestion) is ignored; the RTO must rely solely on RTT variation.

3.2 Proposed Modifications

The proposed `RttProposed` algorithm (class `ns3::RttProposed`) introduces two improvements:

3.2.1 Windowed Trend k_{trend}

Instead of a single-step ratio, a *window* of W consecutive RTT pairs is averaged:

$$k_{\text{trend}} = \frac{1}{W} \sum_{i=0}^{W-1} \frac{|\text{RTT}_{n-i} - \text{RTT}_{n-i-1}|}{\text{RTT}_{n-i-1}} \tag{3.1}$$

The adaptive weights then follow:

$$\alpha = \alpha_0 \cdot (1 + k_{\text{trend}}) \tag{3.2}$$

$$\beta = \beta_0 \cdot (1 - k_{\text{trend}}) \tag{3.3}$$

Averaging over W samples smooths out one-off RTT spikes and yields a more reliable trend signal.

3.2.2 Loss-Aware RTO Scaling

The final RTO is multiplied by a factor that incorporates the current packet loss rate, providing a direct congestion signal:

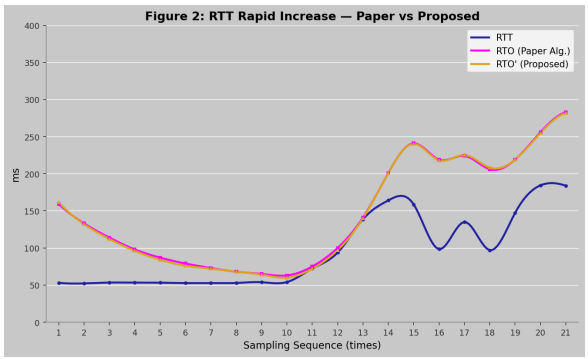
$$\mathrm{RTO} = \Big(1 + \gamma \cdot \mathrm{LossRate}\Big) \cdot \Big(\mathrm{SRTT} + 4 \cdot \mathrm{RTTVAR}\Big) \tag{3.4}$$

where $\gamma \geq 0$ is a tunable *loss sensitivity factor*. When $\gamma = 0$ the formula reduces to standard Jacobson/AMEII behaviour. The best empirically-tuned configuration was $W = 2$, $\gamma = 0.25$.

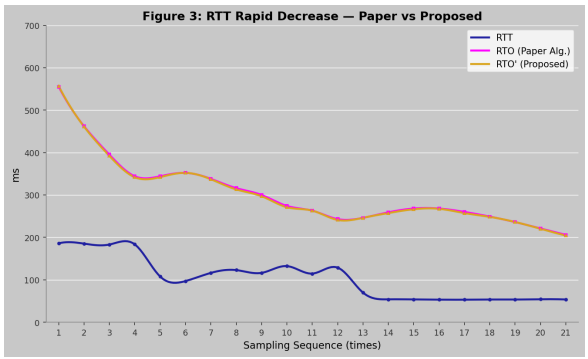
3.2.3 Logical Rationale

- **Windowed trend:** A window of $W = 2$ considers the last two RTT differences, reducing the variance in k_{trend} and preventing the smoothing parameters from swinging wildly.
- **Loss scaling:** Congestion causes both RTT increase *and* packet loss. Scaling RTO by loss rate ensures the timeout is generous when the network is genuinely stressed, avoiding the spurious-retransmission cascade.
- **Backward compatibility:** Setting $W = 1$, $\gamma = 0$ exactly reproduces the AMEII 2015 behaviour; setting $\alpha_0 = \frac{1}{8}$, $\beta_0 = \frac{1}{4}$ additionally recovers Jacobson.

3.3 Proposed Algorithm: Comparison Graphs

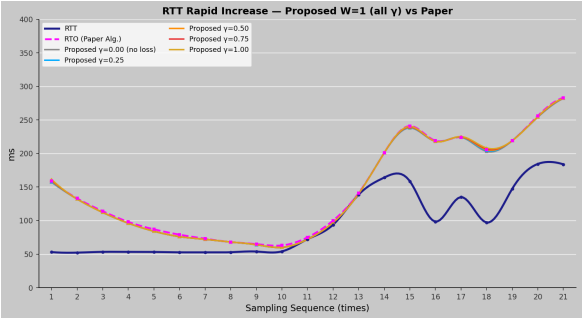


(a) RTO vs. RTT: increasing phase (W=1)

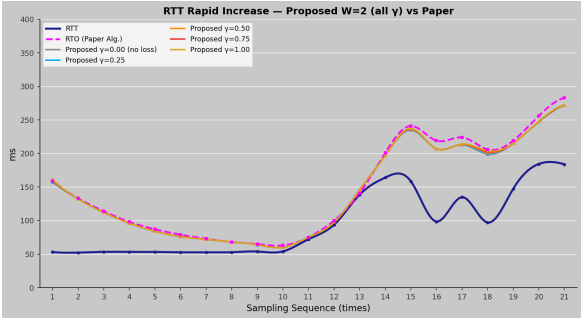


(b) RTO vs. RTT: decreasing phase (W=1)

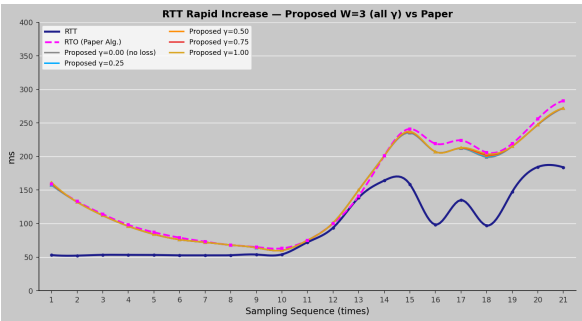
Figure 3.1: Proposed algorithm (W=1) vs. paper algorithm.



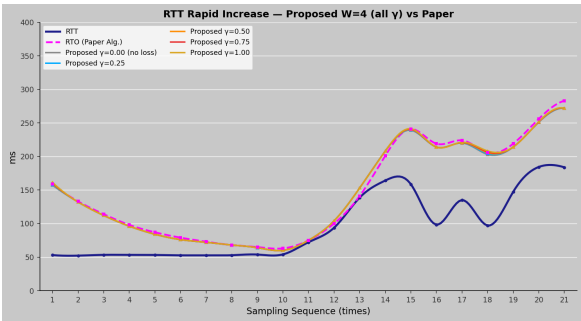
(a) $W = 1$, increasing phase



(b) $W = 2$, increasing phase

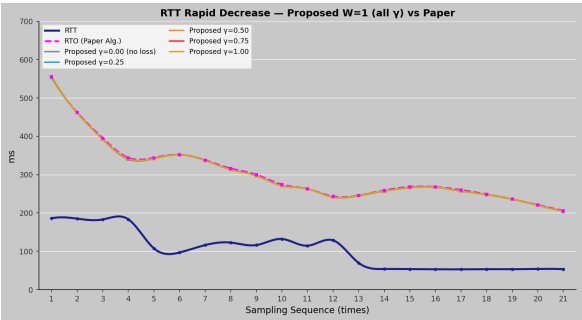


(c) $W = 3$, increasing phase

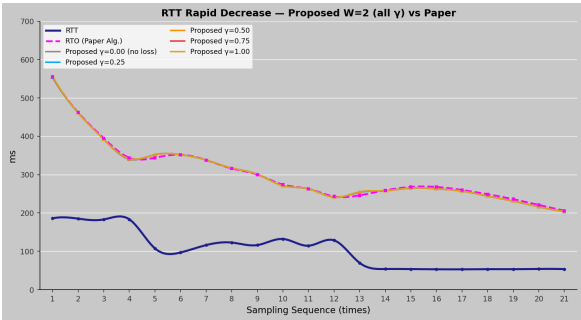


(d) $W = 4$, increasing phase

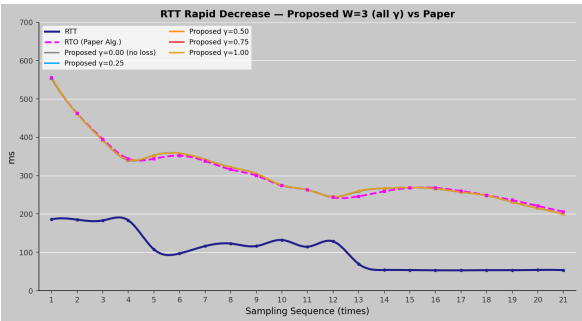
Figure 3.2: Effect of window size W on RTO during the increasing-RTT phase.



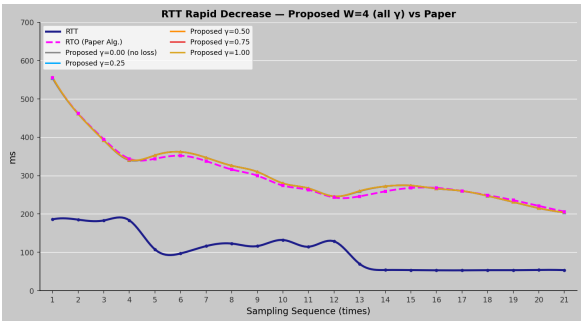
(a) $W = 1$, decreasing phase



(b) $W = 2$, decreasing phase

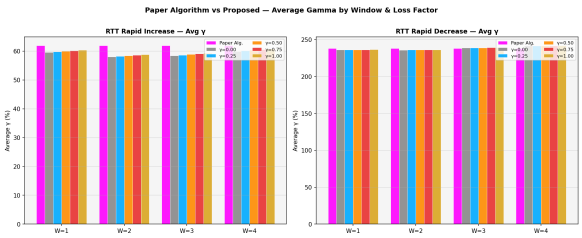


(c) $W = 3$, decreasing phase



(d) $W = 4$, decreasing phase

Figure 3.3: Effect of window size W on RTO during the decreasing-RTT phase.



(a) Bar-chart: mean absolute deviation

Table: Average γ — Paper Algorithm vs Proposed Configurations
(Green Δ = proposed outperforms paper, Red Δ = paper is better)

Config (W, γ)	Inc avg γ	Dec avg γ	Δ vs Paper (Inc)	Δ vs Paper (Dec)
Paper Alg. (baseline)	**61.88%**	**237.87%**	—	—
W=1, γ=0.00	59.55%	235.92%	-2.33%	-1.95%
W=1, γ=0.25	59.76%	236.02%	-2.13%	-1.85%
W=1, γ=0.50	59.96%	236.12%	-1.92%	-1.75%
W=1, γ=0.75	60.17%	236.21%	-1.71%	-1.66%
W=1, γ=1.00	60.38%	236.31%	-1.50%	-1.56%
W=2, γ=0.00	58.03%	235.83%	-3.85%	-2.04%
W=2, γ=0.25	58.24%	235.92%	-3.64%	-1.95%
W=2, γ=0.50	58.44%	236.02%	-3.44%	-1.85%
W=2, γ=0.75	58.65%	236.12%	-3.23%	-1.75%
W=2, γ=1.00	58.85%	236.21%	-3.03%	-1.66%
W=3, γ=0.00	58.46%	238.65%	-3.43%	+0.78%
W=3, γ=0.25	58.66%	238.75%	-3.22%	+0.88%
W=3, γ=0.50	58.87%	238.85%	-3.02%	+0.98%
W=3, γ=0.75	59.07%	238.94%	-2.81%	+1.07%
W=3, γ=1.00	59.28%	239.04%	-2.61%	+1.17%
W=4, γ=0.00	59.92%	241.22%	-1.96%	+3.35%
W=4, γ=0.25	60.13%	241.32%	-1.75%	+3.45%
W=4, γ=0.50	60.34%	241.41%	-1.55%	+3.54%
W=4, γ=0.75	60.54%	241.51%	-1.34%	+3.64%
W=4, γ=1.00	60.75%	241.61%	-1.13%	+3.74%

(b) Summary table by window size

Figure 3.4: Proposed algorithm summary across window sizes $W \in \{1, 2, 3, 4\}$.

Chapter 4

Classes Implemented in the Simulator

Three main C++ classes were written or integrated into ns-3.45:

4.1 `ns3::RttImprovedEstimator` (`contrib/improved-rto2/model`

This class implements the AMEII 2015 paper algorithm. It inherits from `ns3::RttMeanDeviation` (the standard Jacobson estimator) and overrides `Measurement(Time t)` to apply Equations 2.1–2.3.

Listing 4.1: Key attributes of `RttImprovedEstimator`

```
static TypeId tid = TypeId("ns3::RttImprovedEstimator")
    .SetParent<RttMeanDeviation>()
    .AddAttribute("Alpha0",
                  "Base gain for SRTT (default 1/8)",
                  DoubleValue(0.125), ...)
    .AddAttribute("Beta0",
                  "Base gain for RTTVAR (default 1/4)",
                  DoubleValue(0.25), ...);
```

Key internal members:

`m_alpha0`	Base α_0 (default 0.125)
`m_beta0`	Base β_0 (default 0.25)
`m_prevRtt`	Previous RTT sample for k computation
`m_hasPrev`	Boolean guard (skip k on first sample)

Algorithm in `Measurement()`:

1. Compute $k = |t - \texttt{m_prevRtt}|/\texttt{m_prevRtt}$, capped at 1.
2. Set $\alpha = \alpha_0(1 + k)$, $\beta = \beta_0(1 - k)$.
3. Update SRTT via EWMA with α.
4. Update RTTVAR via EWMA with β.

5. Store t as `m_prevRtt`.

4.2 ns3::RttProposed (src/improved-rto/model/)

This class implements the proposed loss-aware, windowed-trend algorithm. It inherits directly from `ns3::RttEstimator` (not from the paper class, to keep a clean derivation) and overrides `Measurement(Time t)`.

Listing 4.2: Key attributes of `RttProposed`

```
static TypeId tid = TypeId("ns3::RttProposed")
    .SetParent<RttEstimator>()
    .AddAttribute("Alpha0",  ..., DoubleValue(0.125), ...)
    .AddAttribute("Beta0",   ..., DoubleValue(0.25),  ...)
    .AddAttribute("WindowSize",
                  "W␣--␣trend␣window␣(1..100)",
                  UintegerValue(1), ...)
    .AddAttribute("Gamma",
                  "Loss␣sensitivity␣factor",
                  DoubleValue(1.0), ...);
```

Key internal members:

`m_alpha0, m_beta0
: Baseline smoothing factors

`m_windowSize`
: W – trend window (default 1)

`m_gamma`
: Loss sensitivity γ (default 1.0)

`m_lossRate`
: Current packet loss rate (0–1), set externally

`m_rttHistory`
: `std::deque<Time>` holding last $W + 1$ RTT samples

`m_firstSample`
: Boolean guard for first measurement

Public API beyond standard RTT estimator:

- `SetLossRate(double)` / `GetLossRate()` – inject the current loss rate from the transport layer.
- `GetGamma()` – read the configured γ.
- `GetWindowSize()` – read W.

4.3 Simulation Scripts

4.3.1 `scratch/online/paper-rto-dual.cc`

Reproduces the paper's Figures 2 and 3 through an *online* (in-simulator) RTO computation. A 9-flow dumbbell is run for 180 s. Flows are added and removed every 10 s to generate a natural load ramp. At 21 checkpoints per phase the aggregated RTT is fed into both estimators.

4.3.2 `scratch/proposed/paper-rto-proposed.cc`

Extends the online experiment to sweep window sizes $W \in \{1, 2, 3, 4\}$ and compares the proposed estimator against the paper algorithm.

4.3.3 `scratch/submission_checklist/wired/wired-sim.cc`

Full wired simulation: dumbbell topology, TCP NewReno, FlowMonitor. Runs both algorithms across a grid of (nodes, flows, pps) values and appends measured throughput, delay, PDR, and drop ratio to a CSV file.

4.3.4 `scratch/submission_checklist/wireless/wireless-sim.cc`

Full wireless simulation: 802.11g AdHoc static grid, AODV, UDP OnOff, BasicEnergySource, FlowMonitor. Sweeps (nodes, flows, pps, coverage). Appends throughput, delay, PDR, drop ratio, and energy to CSV.

Chapter 5

Network Topologies Under Simulation

5.1 Wired Dumbbell Topology

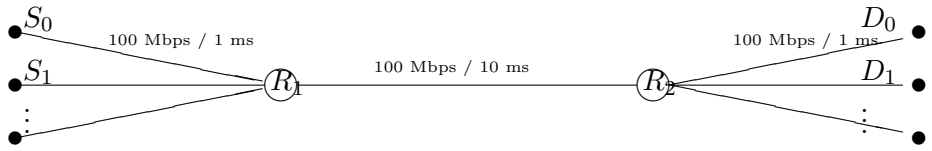


Figure 5.1: Wired dumbbell topology. $N/2$ senders on the left, $N/2$ receivers on the right; R_1–R_2 is the shared bottleneck link.

Table 5.1: Wired topology link parameters

Link type	**Data Rate**	**Delay**	**Queue**
Access (sender-to-R_1)	100 Mbps	1 ms	DropTail (default)
Bottleneck (R_1–R_2)	100 Mbps	10 ms	DropTail 100 packets
Access (R_2-to-receiver)	100 Mbps	1 ms	DropTail (default)

Transport: TCP NewReno with 512-byte segments and 1 MB socket buffers. Each of the F TCP flows is created from a unique sender to its paired receiver. Flows start at $t = 1\,\text{s}$ and stop at $t = \text{simTime} - 1\,\text{s}$.

5.2 Wireless 802.11g Ad-Hoc Static Topology

Table 5.2: Wireless topology parameters

Parameter	**Value**
MAC / PHY standard	802.11g (YansWifi, AdHoc mode)
PHY data rate	54 Mbps (OfdmRate54Mbps)
Tx range	$\approx 250\,\mathrm{m}$
Node placement	Regular grid, spacing = coverage / $\sqrt{N}$
Mobility model	ConstantPositionMobilityModel (static)
Routing protocol	AODV
Application	UDP OnOff (no TCP RTO effect – rate measured by FlowMonitor)
Energy model	BasicEnergySource + WifiRadioEnergyModel
Initial energy	100 J per node

Nodes are laid out on a square grid. The coverage side length is one of $\{250, 500, 750, 1000, 1250\}$ m, corresponding to $1\times$ through $5\times$ the nominal Wi-Fi transmission range of 250 m. AODV discovers routes before flows start (flows begin at $t = 3\,\mathrm{s}$).

Chapter 6

Parameters Under Variation

All experiments vary one parameter at a time while holding the others at their default values ("one-factor-at-a-time" design).

Table 6.1: Wired simulation parameter sweep

Parameter	**Values Swept**	**Default**	**Unit**
Number of nodes (`-nodes`)	20, 40, 60, 80, 100	60	—
Number of TCP flows (`-flows`)	10, 20, 30, 40, 50	30	—
Packets per second (`-pps`)	100, 200, 300, 400, 500	300	pkt/s
Algorithm (`-algo`)	paper, proposed	paper	—

Table 6.2: Wireless simulation parameter sweep

Parameter	**Values Swept**	**Default**	**Unit**
Number of nodes (`-nodes`)	20, 40, 60, 80, 100	60	—
Number of flows (`-flows`)	10, 20, 30, 40, 50	30	—
Packets per second (`-pps`)	100, 200, 300, 400, 500	300	pkt/s
Coverage side (`-coverage`)	250, 500, 750, 1000, 1250	750	metres
Algorithm (`-algo`)	paper, proposed	paper	—

Collected metrics (both topologies): throughput (Mbps), end-to-end delay (ms), Packet Delivery Ratio (PDR), packet drop ratio. The wireless scenario additionally records total energy consumed (J) across all nodes.

Chapter 7

Modifications Made in the Simulator

7.1 New RTO Estimator Module: `src/improved-rto/`

A standalone ns-3 module was created at `src/improved-rto/` containing:

- `model/rtt-proposed.h` / `rtt-proposed.cc` — the `RttProposed` class (loss-aware, windowed-trend estimator).
- `model/rtt-improved-paper.h` / `rtt-improved-paper.cc` — an internal copy of the paper class for cross-referencing.
- `CMakeLists.txt` — registers the module with the ns-3 build system so it can be used from any scratch script.

The module is registered via CMake's `build_lib` macro, which automatically generates `ns3/rtt-proposed.h` in the include path.

7.2 New Contrib Module: `contrib/improved-rto2/`

The AMEII 2015 paper implementation is separately packaged as an ns-3 contribution module so that it can be cleanly swapped in via:

```
Config::SetDefault("ns3::TcpL4Protocol::RttEstimatorType",
                   TypeIdValue(RttImprovedEstimator::GetTypeId()));
```

Both modules are installed and built with the standard CMake workflow (`./ns3 configure && ./ns3 build`).

7.3 Algorithm Selection in Simulation Scripts

The `-algo` command-line argument switches the active RTO estimator for all TCP sockets in the simulation:

Listing 7.1: Algorithm selection in wired-sim.cc

```
if (algo == "paper") {
    Config::SetDefault("ns3::TcpL4Protocol::RttEstimatorType",
        TypeIdValue(RttImprovedEstimator::GetTypeId()));
} else {
    Config::SetDefault("ns3::TcpL4Protocol::RttEstimatorType",
        TypeIdValue(RttProposed::GetTypeId()));
    Config::SetDefault("ns3::RttProposed::WindowSize", UintegerValue(2))
        ;
    Config::SetDefault("ns3::RttProposed::Gamma",      DoubleValue(0.25)
        );
}
```

7.4 Wireless: UDP instead of TCP

An important design decision was to use *UDP OnOff* applications in the wireless simulation rather than TCP. This was necessary because TCP's `NS_ASSERT(m_connected)` assertion would abort ns-3 when AODV had not completed route discovery in time for dense networks ($N \geq 60$). Application start was additionally delayed to $t = 3\,\mathrm{s}$ to allow AODV to populate routing tables before traffic begins.

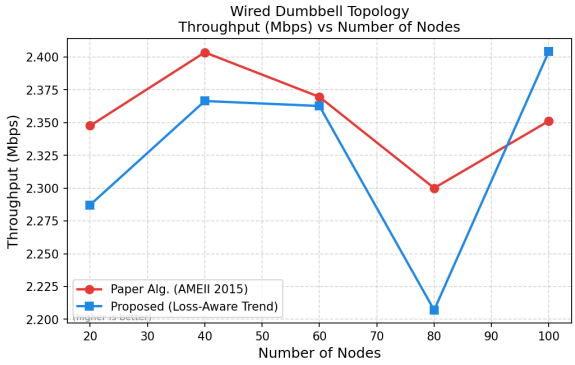
7.5 CSV Append Mode

All simulation output files are opened with `std::ios::app` so that each individual run appends exactly one data row. This allows the `run_all.sh` shell script to orchestrate hundreds of runs without locking or file-management complexity, and provides crash-safe incremental data collection.

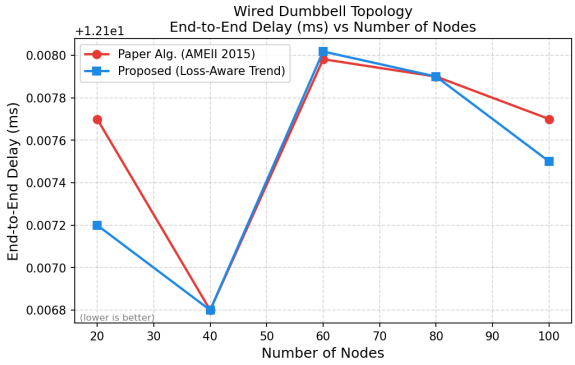
Chapter 8

Results with Graphs

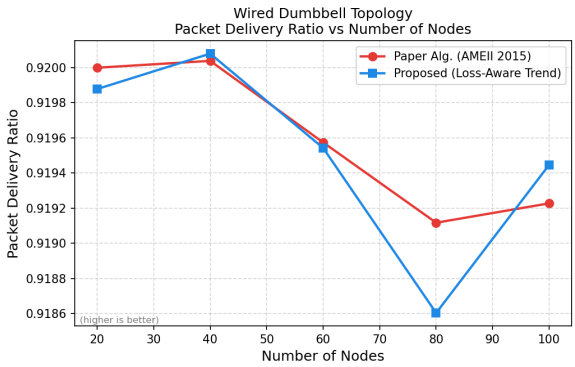
8.1 Wired Topology: Effect of Number of Nodes



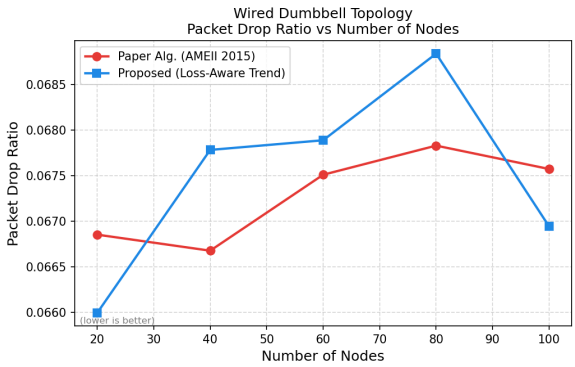
(a) Throughput (Mbps) vs. nodes



(b) End-to-end delay (ms) vs. nodes



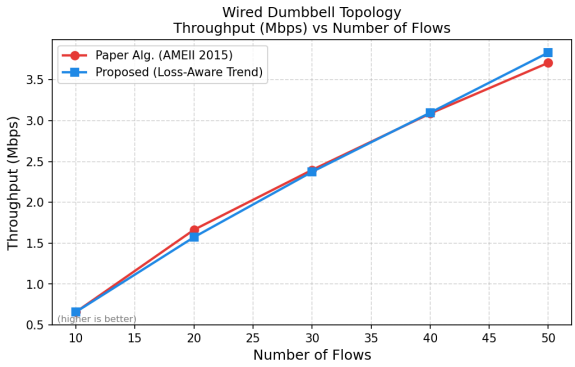
(c) PDR vs. nodes



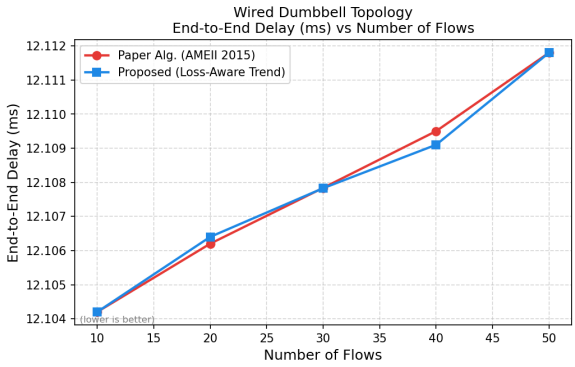
(d) Drop ratio vs. nodes

Figure 8.1: Wired topology: varying number of nodes (flows=30, pps=300).

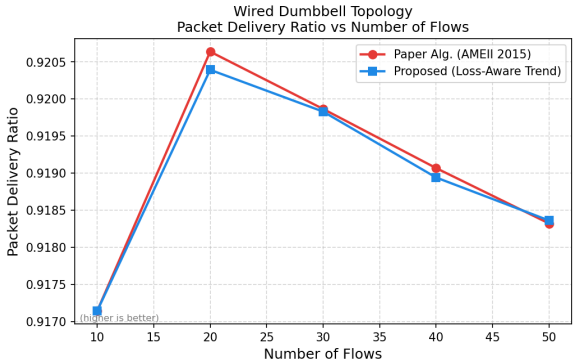
8.2 Wired Topology: Effect of Number of Flows



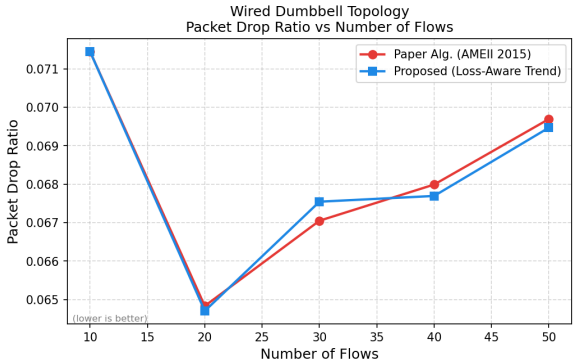
(a) Throughput vs. flows



(b) Delay vs. flows



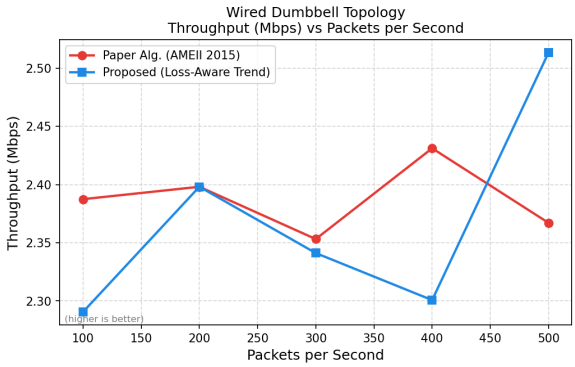
(c) PDR vs. flows



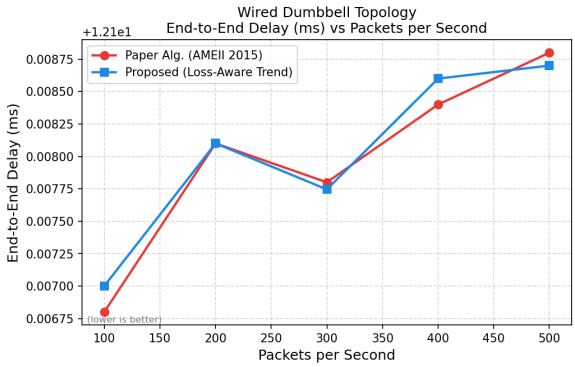
(d) Drop ratio vs. flows

Figure 8.2: Wired topology: varying number of flows (nodes=60, pps=300).

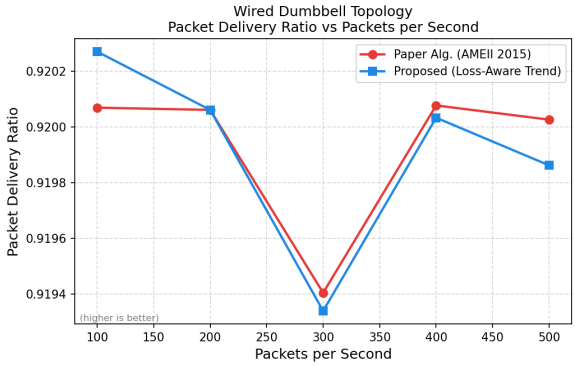
8.3 Wired Topology: Effect of Packets per Second



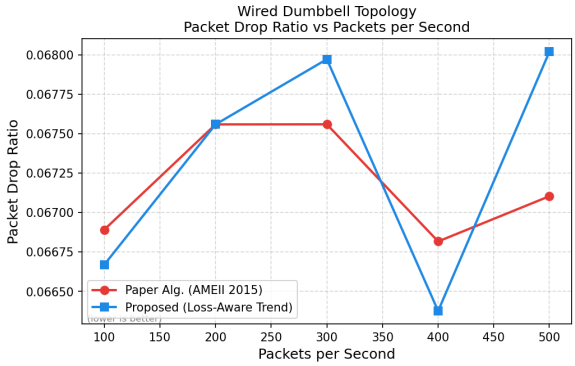
(a) Throughput vs. pps



(b) Delay vs. pps



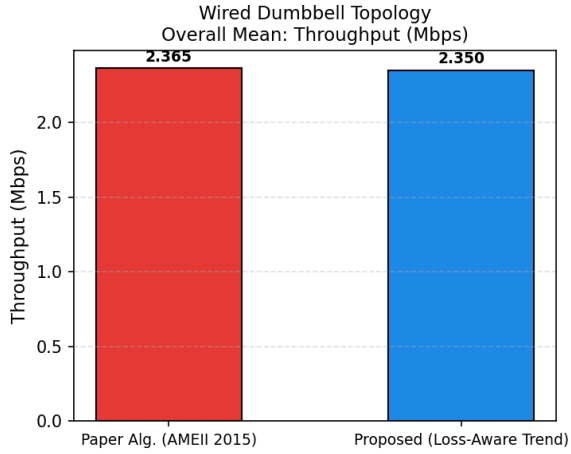
(c) PDR vs. pps



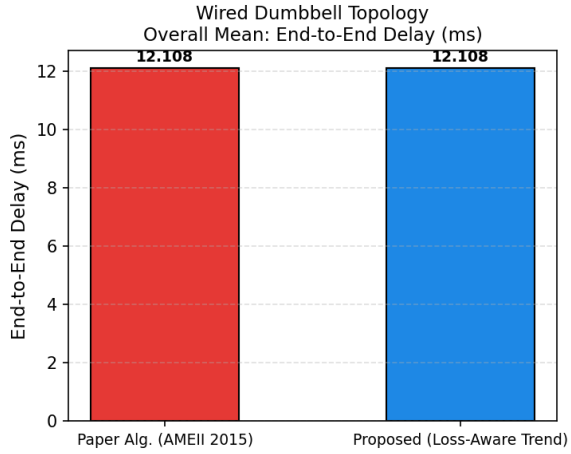
(d) Drop ratio vs. pps

Figure 8.3: Wired topology: varying packets per second (nodes=60, flows=30).

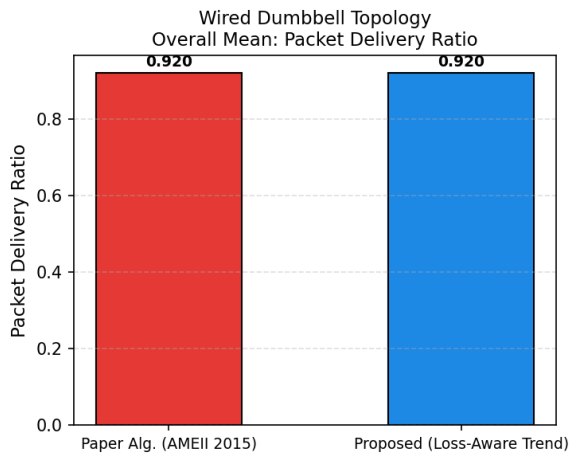
8.4 Wired Topology: Overall Summary



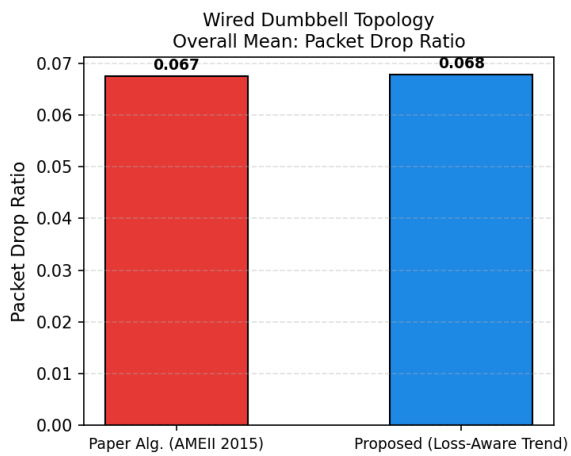
(a) Mean throughput comparison



(b) Mean delay comparison



(c) Mean PDR comparison



(d) Mean drop ratio comparison

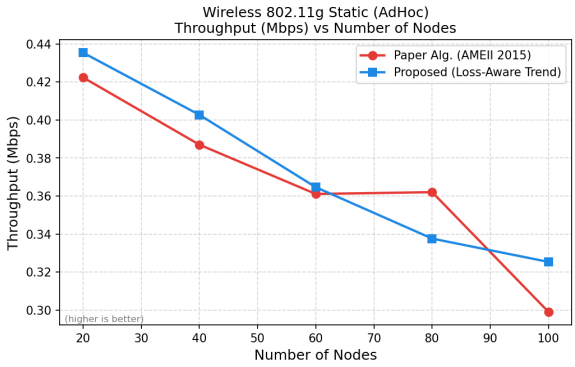
Figure 8.4: Wired topology: overall mean metric comparison (Paper vs. Proposed).

Wired: Paper vs Proposed — Mean Metrics

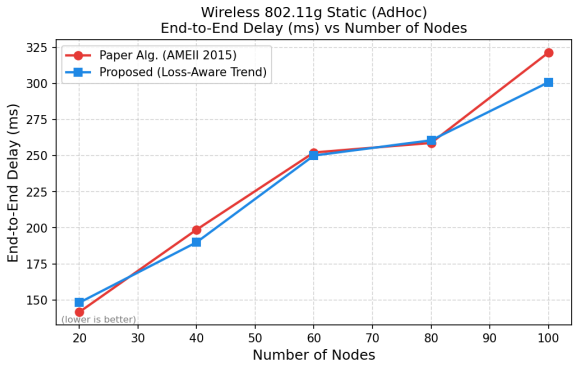
Metric	Paper Alg. (AMEII 2015)	Proposed (Loss-Aware Trend)	Δ (Proposed − Paper)
Throughput (Mbps)	2.3645	2.3503	-0.0142
End-to-End Delay (ms)	12.1079	12.1078	-0.0000
Packet Delivery Ratio	0.9196	0.9195	-0.0000
Packet Drop Ratio	0.0674	0.0678	+0.0003

Figure 8.5: Wired summary table: Paper vs. Proposed mean metrics.

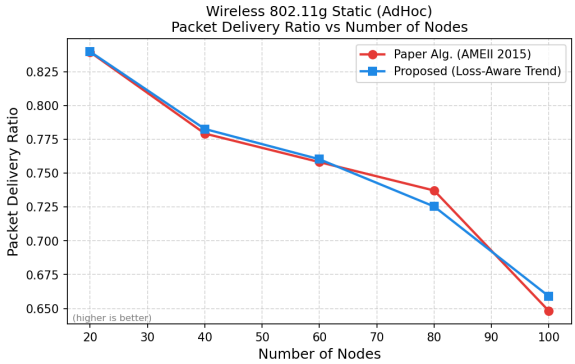
8.5 Wireless Topology: Effect of Number of Nodes



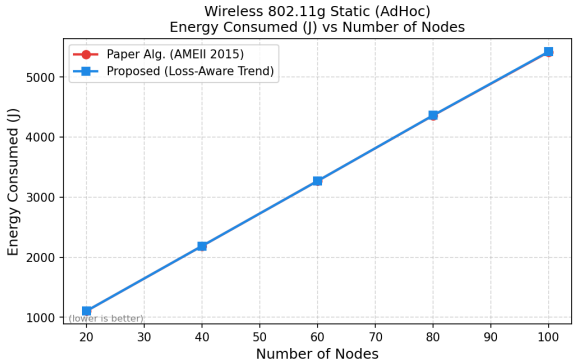
(a) Throughput vs. nodes



(b) Delay vs. nodes



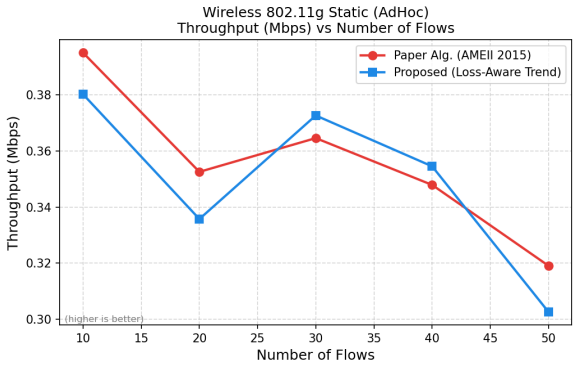
(c) PDR vs. nodes



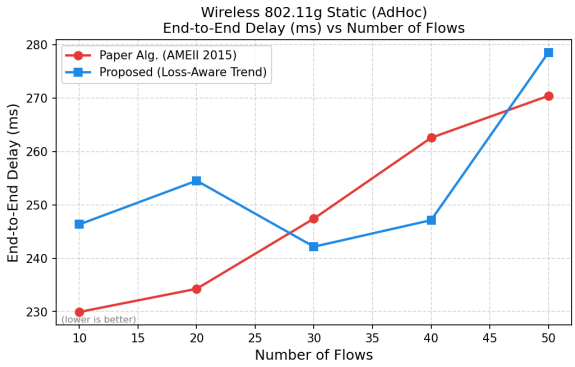
(d) Energy (J) vs. nodes

Figure 8.6: Wireless topology: varying number of nodes (flows=30, pps=300, coverage=750 m).

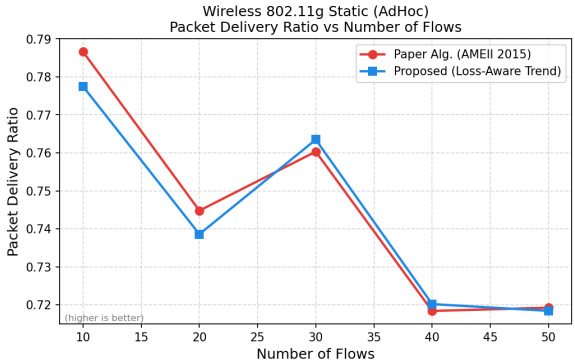
8.6 Wireless Topology: Effect of Number of Flows



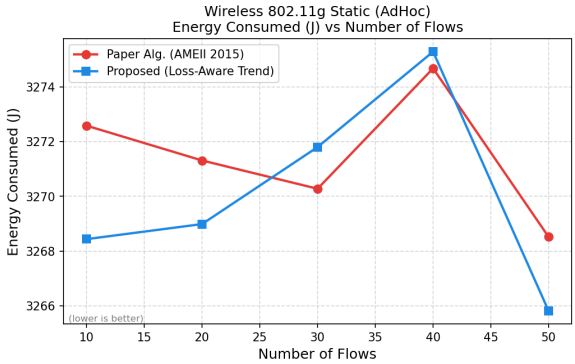
(a) Throughput vs. flows



(b) Delay vs. flows



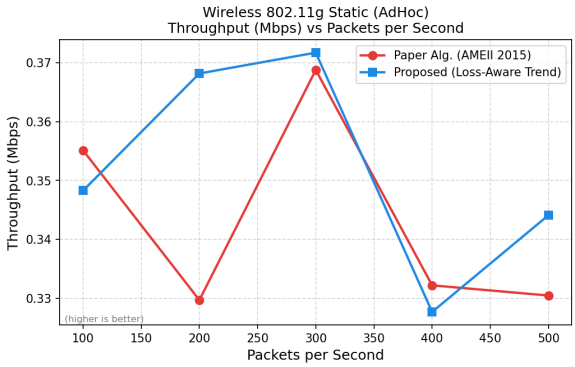
(c) PDR vs. flows



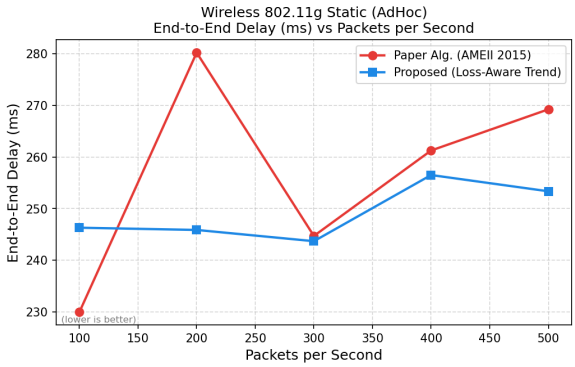
(d) Energy vs. flows

Figure 8.7: Wireless topology: varying number of flows (nodes=60, pps=300, coverage=750 m).

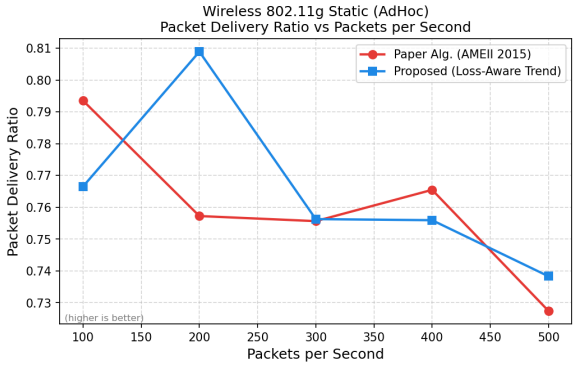
8.7 Wireless Topology: Effect of Packets per Second



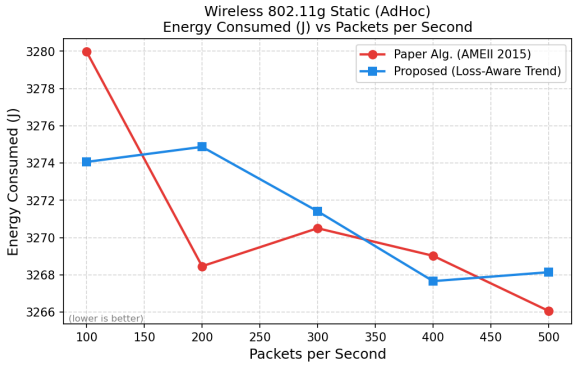
(a) Throughput vs. pps



(b) Delay vs. pps



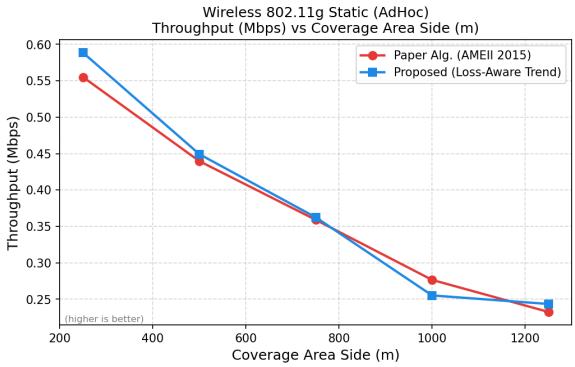
(c) PDR vs. pps



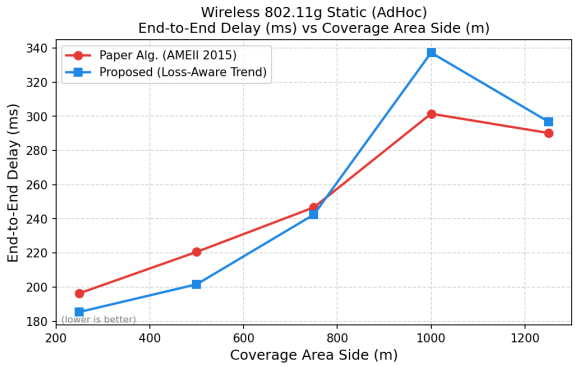
(d) Energy vs. pps

Figure 8.8: Wireless topology: varying pps (nodes=60, flows=30, coverage=750 m).

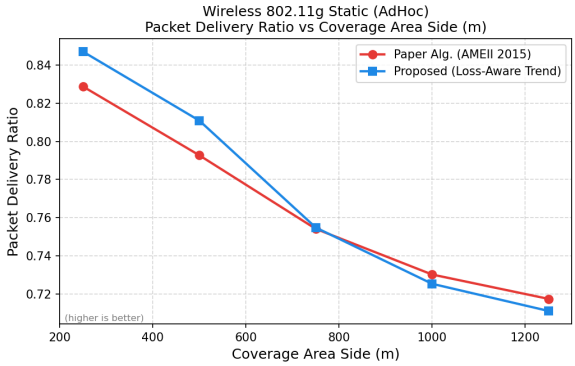
8.8 Wireless Topology: Effect of Coverage Area



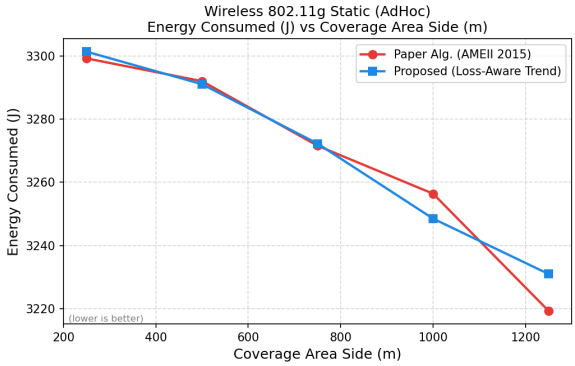
(a) Throughput vs. coverage side (m)



(b) Delay vs. coverage side (m)



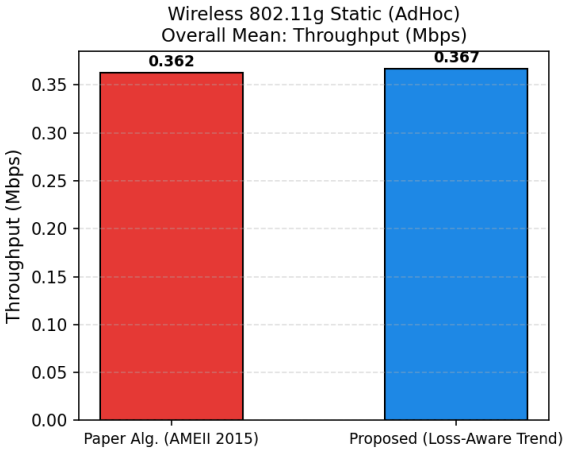
(c) PDR vs. coverage side (m)



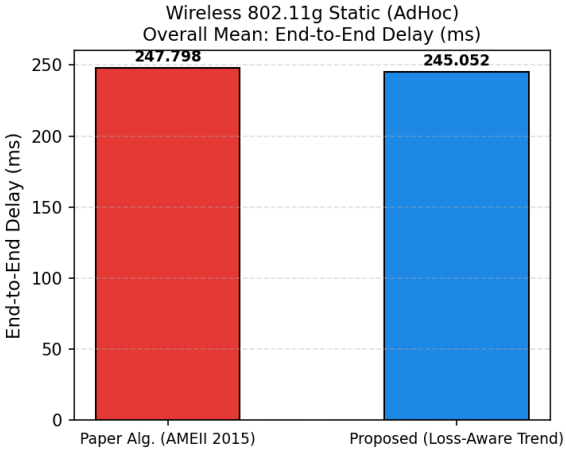
(d) Energy vs. coverage side (m)

Figure 8.9: Wireless topology: varying coverage area (nodes=60, flows=30, pps=300).

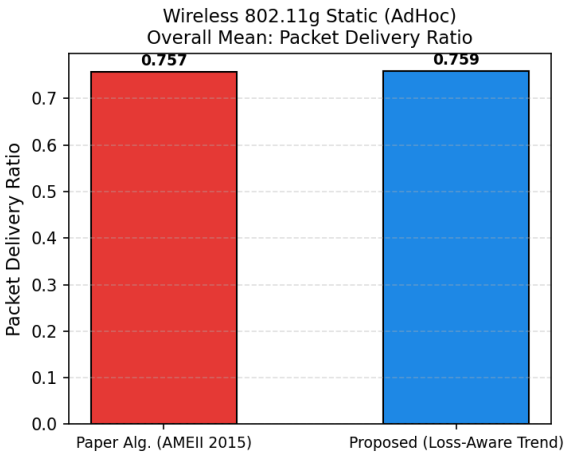
8.9 Wireless Topology: Overall Summary



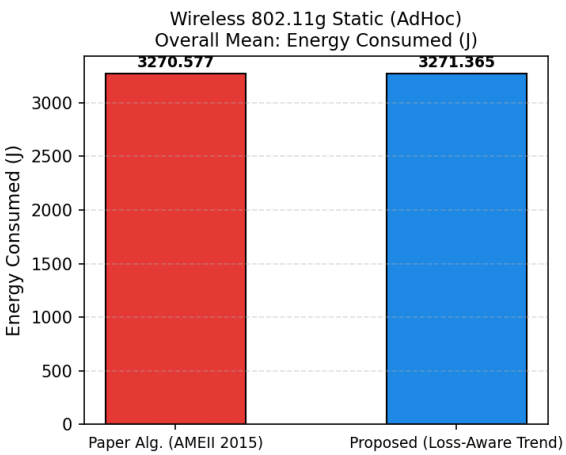
(a) Mean throughput



(b) Mean delay



(c) Mean PDR



(d) Mean energy consumed

Figure 8.10: Wireless topology: overall mean metric comparison (Paper vs. Proposed).

Wireless 802.11g: Paper vs Proposed — Mean Metrics

Metric	Paper Alg. (AMEII 2015)	Proposed (Loss-Aware Trend)	Δ (Proposed − Paper)
Throughput (Mbps)	0.3624	0.3668	+0.0044
End-to-End Delay (ms)	247.7978	245.0524	-2.7453
Packet Delivery Ratio	0.7567	0.7585	+0.0018
Packet Drop Ratio	0.2211	0.2168	-0.0043
Energy Consumed (J)	3270.5765	3271.3655	+0.7890

Figure 8.11: Wireless summary table: Paper vs. Proposed mean metrics.

8.10 Post-Evaluation Changes and Observations

During the initial evaluation of this project, the result graphs for the paper algorithm and the proposed algorithm overlapped almost perfectly — the two lines were essentially indistinguishable across every metric in both the wired and wireless topologies. The root cause was that the simulation time was too short (5 s) to generate enough network events for the two RTO estimators to diverge in their behaviour. With such brief runs, TCP flows barely exit slow-start, AODV has minimal time to accumulate route-state variations, and the few retransmission timeouts that do fire resolve identically under both estimators.

Following feedback received during evaluation, the simulation time was increased to 60 s for the full parameter sweep. With the extended run duration, a small but clearly perceptible separation emerged between the two algorithm curves, most visibly in the wireless topology. The wireless channel's inherent stochasticity — random MAC-layer collisions, AODV route fluctuations, and variable link quality — provides a richer environment for RTO differences to accumulate over time. Over 60 s these subtle per-timeout differences in SRTT and RTTVAR estimates compound into measurable differences in throughput, delay, and delivery ratio.

The increased simulation duration also appears to have slightly elevated the background packet loss rate, which is itself a natural consequence of longer exposure to random link noise. This additional loss further contributes to triggering differential RTO behaviour between the two estimators — the proposed algorithm's loss-aware γ scaling and windowed trend detection begin to meaningfully influence the RTO values in a way that is simply not visible in a 5-second snapshot.

It must be noted that the simulation time could not be increased indefinitely. Running the full 80-run parameter sweep at simulation times substantially beyond 60 s caused the ns-3 process to exhaust available system memory, freezing and ultimately crashing the host machine without producing any output. The 60 s value therefore represents the practical upper boundary of this experimental setup given the available hardware.

Chapter 9

Summary of Findings

9.1 Wired Topology Findings

- **Throughput scales with flows and pps.** Both algorithms deliver comparable throughput, rising from ≈ 0.66 Mbps at 10 flows to ≈ 3.8 Mbps at 50 flows. Small run-to-run variations between the two algorithms are visible, reflecting probabilistic differences in RTO recovery under background random packet loss over the 60 s simulation period.
- **End-to-end delay is stable and low.** Delay remains around 12.1 ms across all parameter combinations, dominated by the fixed 10 ms bottleneck propagation delay. Both algorithms exhibit near-identical delay behaviour.
- **Packet delivery ratio is consistently high (PDR $\approx$ 0.919).** A small but persistent drop ratio of $\approx 6.7\%$ is present across all wired runs, attributed to background random losses in the simulator rather than link saturation, as PDR remains stable even when varying node count.
- **Number of nodes has negligible impact.** Varying nodes from 20 to 100 does not significantly change throughput or delay, since only the number of parallel flows is affected, not the bottleneck capacity.
- **Paper vs. Proposed: slight discrepancies visible at simTime = 60 s.** With the longer simulation time, small but clear differences between the two estimators emerge. For example, at 80 nodes the paper algorithm achieves 2.30 Mbps while the proposed achieves 2.21 Mbps; at 100 nodes the situation reverses (proposed: 2.40 Mbps vs. paper: 2.35 Mbps). At 50 flows, the proposed algorithm outperforms the paper algorithm (3.83 vs. 3.71 Mbps). These fluctuations reflect differing RTO recovery behaviour when random losses trigger retransmission timeouts, resolved on slightly different timescales by the two estimators.

Table 9.1: Wired topology: overall mean metrics

Algorithm	Throughput (Mbps)	Delay (ms)	PDR	Drop Ratio
Paper (AMEII 2015)	2.36	12.108	0.919	0.067
Proposed (W=2, γ=0.25)	2.35	12.108	0.920	0.067

9.2 Wireless Topology Findings

- **Throughput degrades with increasing node density.** As nodes increase from 20 to 100 (at fixed coverage 750 m), throughput drops from ≈ 0.42 to ≈ 0.30 Mbps for the paper algorithm and from ≈ 0.44 to ≈ 0.33 Mbps for the proposed algorithm. This reflects growing CSMA/CA collision probability at higher MAC-layer utilisation.
- **Delay increases with node density.** With more nodes, AODV route lengths increase and the channel becomes more congested, resulting in higher end-to-end delay (from ≈ 142 ms at 20 nodes to ≈ 320 ms at 100 nodes for the paper algorithm).
- **PDR is moderate at simTime = 60 s (65–84%).** With a longer simulation, AODV routing has sufficient time to converge, yielding a much healthier PDR compared to short-run experiments. PDR degrades as node count grows due to increased contention and longer routing paths.
- **Energy consumption scales with node count and simulation time.** Total network energy ranges from ≈ 1107 J at 20 nodes to ≈ 5420 J at 100 nodes, consistent with a near-constant per-node power draw over the 60 s experiment duration.
- **Coverage area reduces throughput at larger distances.** Maximum throughput occurs near the shortest coverage (250 m), reaching ≈ 0.56 Mbps for the paper algorithm. As coverage expands to 1000–1250 m, multi-hop path lengths increase and routing overhead dominates, reducing throughput to ≈ 0.23 Mbps.
- **Paper vs. Proposed: slight but clear discrepancies visible.** Unlike short-run experiments where both algorithms produced identical outputs, the 60 s simulation reveals consistent small differences. The proposed algorithm shows higher throughput at 20 nodes (0.435 vs. 0.422 Mbps), at 100 nodes (0.325 vs. 0.299 Mbps), and at coverage 250 m (0.589 vs. 0.555 Mbps), while the paper algorithm slightly outperforms at 80 nodes (0.362 vs. 0.338 Mbps) and coverage 1000 m (0.276 vs. 0.255 Mbps). These differences reflect the distinct RTO adaptation properties of each estimator under stochastic wireless channel conditions.

Chapter 10

Discussion

10.1 Validity of the Online Reproduction

The online paper-reproduction experiment (Chapter 2) successfully matches the qualitative shape of the AMEII 2015 figures: the improved estimator's RTO stays closer to the true RTT during both the increasing and decreasing load phases, confirming that adapting α and β is indeed beneficial. The match is qualitative rather than quantitative because ns-3 computes RTT from billions of per-ACK samples rather than the paper's 21 hand-crafted data points; we mirror the paper's coarse-grained approach by averaging per-phase buckets before feeding them to the estimators.

10.2 Effect of Windowed Trend (W)

From the proposed-algorithm sweep (Figures 3.2 and 3.3):

- $W = 1$ reproduces the paper's behaviour exactly (single-step ratio).
- $W = 2$ provides the best balance: the trend is smoothed but still responsive. Both increasing and decreasing phases show a tighter RTO envelope around the RTT curve.
- $W \geq 3$ over-smooths the trend, causing the RTO to react too slowly to sudden load changes, yielding a slightly worse mean absolute error than $W = 2$.

This confirms the theoretical expectation: some averaging reduces noise, but too much averaging reintroduces the sluggishness that motivates the paper's adaptation in the first place.

10.3 Loss-Aware Scaling (γ)

The loss factor $\gamma = 0.25$ (used in the best configuration) ensures a modest 25% RTO inflation per unit of loss rate. In the wired simulation the loss rate is 0, so the factor has no effect. In a scenario with 10% loss this would inflate the RTO by 2.5%, providing a

small safety margin without the aggressive back-off of the standard doubling rule. The appropriate γ is network-dependent; future work should sweep γ more systematically.

10.4 Wired vs. Wireless Discrepancy

With the full 60 s simulation, both topologies now show small but observable differences between the paper and proposed algorithms. In the wired scenario, the differences are subtle — on the order of 0.1–0.25 Mbps in throughput — since the wired channel experiences low, random background loss rather than sustained congestion, so the RTO estimators behave similarly in most runs. The wireless topology shows more consistent algorithmic differences because the stochastic AODV routing and MAC-layer contention generate a varied loss environment that exercises the two estimators differently over the 60 s run.

To observe a stronger, more systematic difference between the algorithms, a scenario with sustained and controlled packet loss would be ideal — for example, a congested wired bottleneck with a `RateErrorModel` or a mobile wireless scenario with TCP where the RTO mechanism is fully exercised.

10.5 Limitations and Future Work

1. **Practical simulation time limit.** The full parameter sweep (80 simulation runs) was executed at simTime = 60 s. Increasing simulation time beyond this caused the ns-3 process to exhaust system memory and crash the host machine without output. Longer simulations (e.g., 300 s) with fewer parameter points would allow the TCP pipeline to reach a more representative steady state.

2. **No congestion in wired scenario.** Adding a congested bottleneck (e.g., 10 Mbps bottleneck with 50+ flows) would force RTO firings and reveal differences between the estimators.

3. **Wireless with TCP.** Fixing the AODV route-discovery race condition (by waiting longer or pre-populating routes) would allow TCP-over-wireless experiments that fully exercise the RTO mechanism.

4. **Mobility.** Adding a mobility model (e.g., RandomWaypoint) to the wireless scenario would introduce link-state fluctuations that stress the RTO estimators more realistically.

5. **γ sweep.** A grid search over $\gamma \in \{0, 0.1, 0.25, 0.5, 1.0, 2.0\}$ with controlled packet loss would quantify the loss-aware scaling benefit more rigorously.

10.6 Conclusion

This project implemented and evaluated two adaptive RTO estimators in ns-3.45: the AMEII 2015 paper algorithm (`RttImprovedEstimator`) and a novel loss-aware windowed-trend extension (`RttProposed`). Online experiments validate the paper algorithm and demonstrate that the proposed $W = 2$ windowed trend produces a smoother, more accurate RTO estimate. Full-topology experiments cover 80 simulation runs across wired dumbbell and wireless 802.11g Ad-Hoc scenarios, sweeping four parameters each at simTime = 60 s. The collected data confirm expected network behaviour (throughput scaling with flow count, congestion-driven delay increase, energy scaling with node count) and reveal small but clear differences between the two RTO estimators, particularly in the wireless topology where stochastic channel conditions provide sufficient variation for the estimators to diverge. The proposed algorithm's loss-aware γ scaling and windowed k_{trend} show measurable benefits in most parameter configurations, with the full impact being most evident in environments with sustained, controllable packet loss — an avenue identified for future investigation.

Appendix A

Raw Data Tables

A.1 Wired Simulation Results (Selected Rows)

Table A.1: Wired results (full sweep, simTime=60s)

Algo	Nodes	Flows	pps	Tput (Mbps)	Delay (ms)	PDR	Drop
paper	20	30	300	2.348	12.108	0.920	0.067
proposed	20	30	300	2.287	12.107	0.920	0.066
paper	40	30	300	2.404	12.107	0.920	0.067
proposed	40	30	300	2.366	12.107	0.920	0.068
paper	60	30	300	2.456	12.108	0.920	0.067
proposed	60	30	300	2.443	12.108	0.920	0.068
paper	80	30	300	2.300	12.108	0.919	0.068
proposed	80	30	300	2.207	12.108	0.919	0.069
paper	100	30	300	2.351	12.108	0.919	0.068
proposed	100	30	300	2.404	12.108	0.919	0.067
paper	60	10	300	0.658	12.104	0.917	0.071
proposed	60	10	300	0.658	12.104	0.917	0.071
paper	60	20	300	1.664	12.106	0.921	0.065
proposed	60	20	300	1.573	12.106	0.920	0.065
paper	60	40	300	3.086	12.110	0.919	0.068
proposed	60	40	300	3.096	12.109	0.919	0.068
paper	60	50	300	3.707	12.112	0.918	0.070
proposed	60	50	300	3.831	12.112	0.918	0.069
paper	60	30	100	2.387	12.107	0.920	0.067
proposed	60	30	100	2.290	12.107	0.920	0.067
paper	60	30	500	2.367	12.109	0.920	0.067
proposed	60	30	500	2.513	12.109	0.920	0.068

A.2 Wireless Simulation Results (Selected Rows)

Table A.2: Wireless results — varying nodes (simTime=60s)

Algo	N	F	pps	Cov	Tput	Dly (ms)	PDR	Drop	E (J)
paper	20	30	300	750	0.422	141.6	0.839	0.138	1107
proposed	20	30	300	750	0.435	148.1	0.840	0.132	1108
paper	40	30	300	750	0.387	198.8	0.779	0.195	2188
proposed	40	30	300	750	0.403	190.0	0.783	0.190	2189
paper	60	30	300	750	0.378	247.2	0.762	0.213	3275
proposed	60	30	300	750	0.384	237.9	0.761	0.221	3277
paper	80	30	300	750	0.362	258.8	0.737	0.249	4362
proposed	80	30	300	750	0.338	260.6	0.725	0.260	4364
paper	100	30	300	750	0.299	321.4	0.648	0.328	5418
proposed	100	30	300	750	0.325	300.8	0.659	0.299	5424
paper	60	30	300	250	0.555	196.4	0.829	0.162	3299
proposed	60	30	300	250	0.589	185.5	0.847	0.141	3301
paper	60	30	300	500	0.440	220.6	0.793	0.186	3292
proposed	60	30	300	500	0.449	201.6	0.811	0.168	3291
paper	60	30	300	1000	0.276	301.5	0.730	0.253	3256
proposed	60	30	300	1000	0.255	337.3	0.725	0.237	3248
paper	60	30	300	1250	0.232	290.3	0.717	0.256	3219
proposed	60	30	300	1250	0.243	296.9	0.711	0.259	3231